

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

## 단원 2 : 머신러닝

DAY1. 머신러닝 학습방법과 데이터 관리

# 목 차

|                        |       |    |
|------------------------|-------|----|
| 1. <u>머신러닝 개발환경 준비</u> | ..... | 3  |
| 2. <u>PyTorch 소개</u>   | ..... | 14 |
| 3. <u>머신러닝 기본 개념</u>   | ..... | 16 |
| 4. <u>데이터 전처리</u>      | ..... | 34 |
| 5. <u>지도학습 알고리즘</u>    | ..... | 37 |
| 6. <u>비지도학습 알고리즘</u>   | ..... | 44 |

# 1. 머신러닝 개발환경 준비

## (1) 파이참(PyCharm) 설치

- 다음 주소를 방문
  - [www.jetbrains.com/pycharm/download](http://www.jetbrains.com/pycharm/download)
  - Community 버전 선택 (무료)
  - 약 300MB 크기
- 설치 옵션
  - Create Associations (.py) 선택
  - Update PATH 선택 (재부팅 필요)

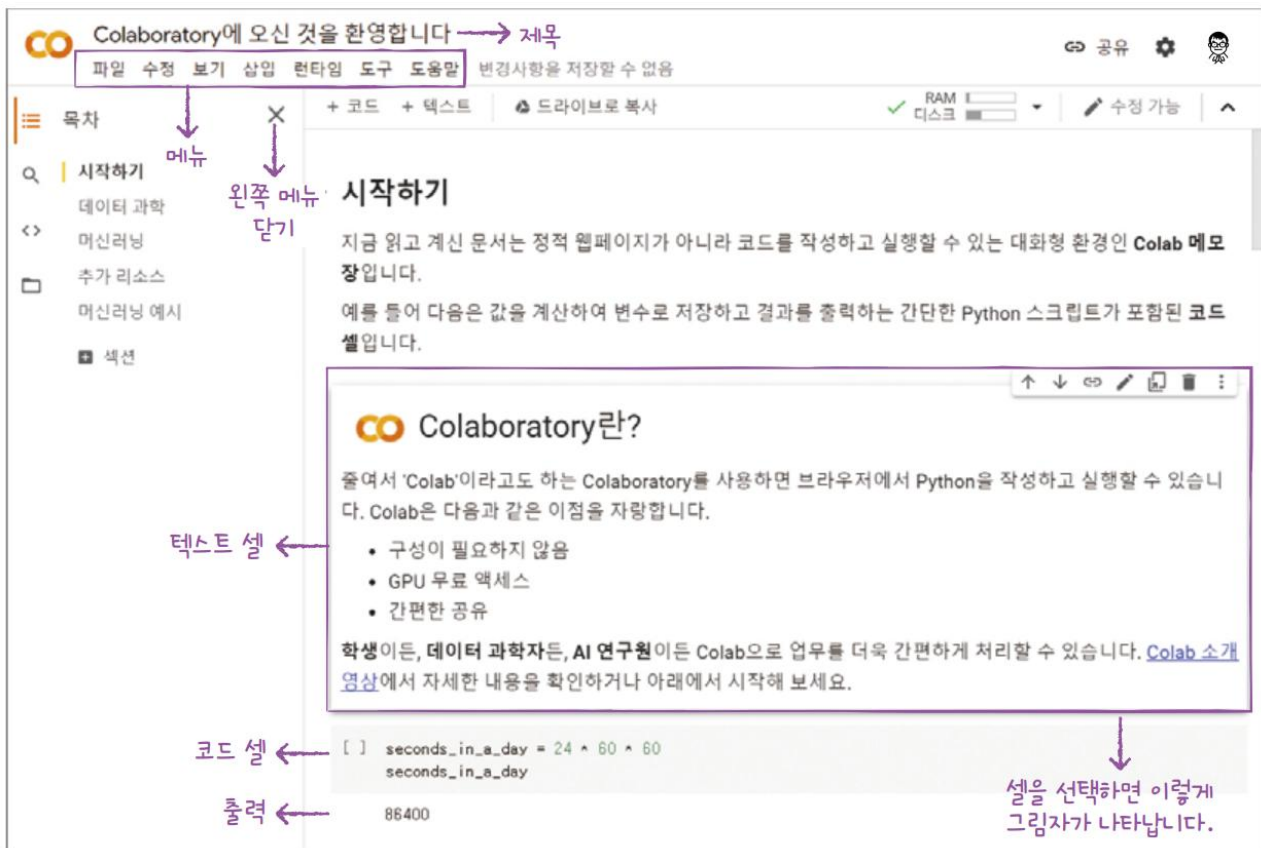
## (1) 새 파이썬 프로젝트 만들기

- 새 파이썬 파일 생성
  - 왼쪽 창에서 hello-torch 프로젝트 선택
  - 오른쪽 마우스 → New → Python File 선택
  - Hello-torch로 명명
- pytorch package 설치 방법
  - File → Settings → Project: hello-torch 선택
  - Python Interpreter 선택
  - 오른쪽에서 "+" 선택: 새로운 화면이 뜸
  - 위에 검색창에서 pytorch (소문자) 검색
  - 왼쪽 밑 Install Package 클릭

# 구글 코랩(Google Colab)으로 머신러닝 시작하기

먼저 웹 서핑이 가능한 컴퓨터를 켜고 구글 코랩에 접속합니다. 기존에 사용하던 구글 계정을 이용하시거나 없다면 새로 계정을 만들어주세요.

구글 코랩 주소: <https://colab.research.google.com/>



코랩은 웹 브라우저에서 텍스트와 프로그램 코드를 자유롭게 작성할 수 있는 온라인 에디터라고 생각하면 쉽습니다. 이런 코랩 파일을 노트북 Notebook 혹은 코랩 노트북이라고 부릅니다.

화면 맨 위에는 노트북의 제목이 쓰여 있습니다. 코랩 기본 페이지의 제목은 'Colaboratory에 오신 것을 환영합니다'입니다. 노트북의 제목은 파일 이름으로도 사용됩니다. 제목 아래 메뉴가 있습니다. 왼쪽에는 현재 페이지의 목차를 보여 줍니다. 노트북에서 셀 cell은 코드 또는 텍스트의 덩어리라고 보면 됩니다. 노트북은 보통 여러 개의 코드 셀과 텍스트 셀로 이루어집니다. 키보드의 위아래 키를 누르면 셀 사이를 이동할 수 있습니다. 물론 마우스로

클릭해서 이동할 수도 있습니다. 선택된 셀은 그림자가 나타나기 때문에 쉽게 알아챌 수 있습니다.

### 여기서 잠깐) 코랩(Colab)은 무엇인가요?

코랩은 웹 브라우저에서 텍스트와 프로그램 코드를 자유롭게 작성할 수 있는 일종의 온라인 텍스트 에디터입니다. 좀 더 있어 보이는 말로 표현하자면 클라우드 기반의 주피터 노트북 개발 환경입니다. 머신러닝은 컴퓨터 사양이 중요한데, 구글 코랩은 나의 느린 컴퓨터 대신에 CPU와 램을 제공해주기 때문에 컴퓨터 성능과 상관없이 프로그램을 실행할 수 있습니다.

당장 코랩에서 중요한 점은 **코랩 파일을 노트북**이라고 부른다는 것과 **코드를 입력하는 곳이 코드 셀**이라는 점입니다. 그리고 코랩에서 사용할 수 있는 프로그래밍 언어는 **'파이썬'**입니다.

### 텍스트 셀

셀 cell은 코랩에서 실행할 수 있는 최소 단위입니다. 즉 셀 안에 있는 내용을 한 번에 실행하고 그 결과를 노트북에 나타냅니다. 하지만 텍스트 셀은 코드처럼 실행되는 것이 아니기 때문에 자유롭게 사용해도 괜찮습니다. 셀 하나에 아주 긴 글을 써도 되고 여러 셀에 나누어 작성해도 괜찮습니다.

The screenshot shows the Google Colaboratory web interface. At the top, there is a toolbar with icons for undo, redo, save, and other functions. Below the toolbar, there are two cells. The first cell is a text cell, indicated by a purple arrow pointing to it with the label '텍스트 입력' (Text Input). It contains HTML code for a Colaboratory logo and a heading 'Colaboratory란?'. The second cell is a code cell, indicated by a purple arrow pointing to it with the label '바로 확인!' (Check immediately!). It contains a message in Korean about using Colaboratory for machine learning and deep learning, and a list of features: '구성 필요하지 않음' (No setup required), 'GPU 무료 액세스' (Free GPU access), and '간편한 공유' (Easy sharing).

텍스트 셀에서는 HTML 과 마크다운 Markdown 을 혼용해서 사용할 수 있습니다. 왼쪽 창에서 텍스트를 수정하면 오른쪽 미리 보기 창에서 수정된 결과를 바로 볼 수 있습니다.<h1> 태그 아래에 임의의 텍스트를 추가하고 미리 보기 창에 나타나는 결과를 확인해 보세요.

## 코드 셀

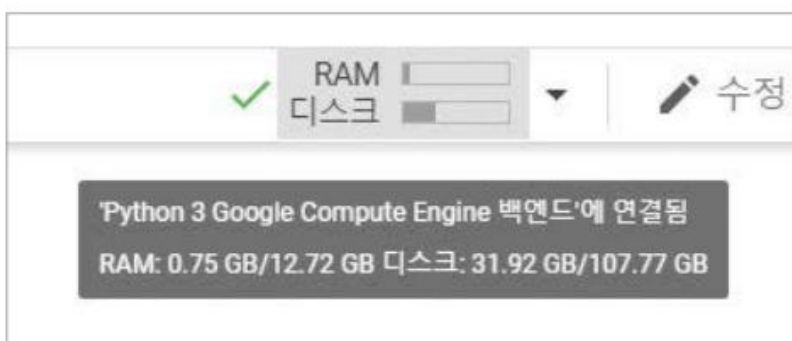
'Colaboratory 에 오신 것을 환영합니다' 노트북의 세 번째 셀이 코드 셀입니다. 코드 셀로 이동하면 코드와 결과가 함께 선택됩니다.

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

그리고 코드 블록의 좌측 버튼 을 클릭하면 86400 이라고 나오면서, 온라인 에디터가 잘 동작한다는 걸 확인할 수 있습니다.

## 노트북

코랩은 구글이 대화식 프로그래밍 환경인 주피터(Jupyter)를 커스터마이징한 것입니다. 파이썬 지원으로 시작한 주피터 프로젝트는 최근에는 다른 언어도 지원합니다. 주피터 프로젝트의 대표 제품이 바로 노트북 Notebook 입니다. 흔히 주피터 노트북이라고 부릅니다.이렇게 입력하고 실행하면 우측 상단에 RAM 과 디스크 표시가 나타납니다. 여기에 마우스 포인터를 갖다 두면 그림처럼 상세 정보를 얻을 수 있습니다.



이 노트북은 구글 클라우드의 컴퓨트 엔진 Compute Engine 에 연결되어 있습니다. 이 서버의 메모리는 약 12 기가이고 디스크 공간은 100 기가입니다. 구글 계정만 있으면 코랩 노트북을 사용해 무료로 가상 서버를 활용할 수 있습니다. 놀랍지 않나요?

다만, 무료로 사용할 때는 노트북을 최대 5 개만 동시에 사용할 수 있습니다. 그리고 노트북 하나를 12 시간 이상 연결할 수 없습니다.

### 여기서 잠깐) 코랩 노트북이 5 개면, 새로운 노트북은 어떻게 만드나요?

코랩 노트북이 5 개 이상 실행 중이라면 실행 중인 웹 브라우저 창을 닫거나, [런타임] → [세션 관리] 메뉴를 선택하여 실행 중인 노트북을 종료할 수 있습니다.

| 활성 세션                                                                                                        |        |         |    |
|--------------------------------------------------------------------------------------------------------------|--------|---------|----|
| 제목                                                                                                           | 마지막 실행 | 사용된 RAM |    |
|  Untitled0.ipynb<br>현재 세션 | 0분 전   | 0.78 GB | 종료 |

### 새 노트북 만들기

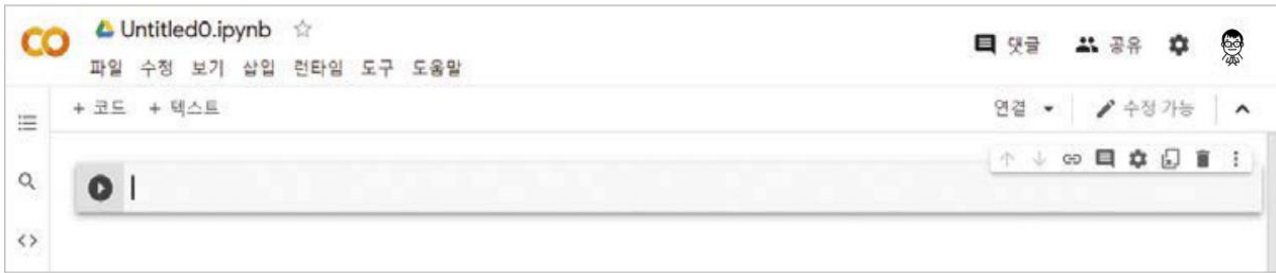
코랩에서 새로운 노트북을 만들고 저장해보겠습니다.

**01** [파일] – [새 노트]를 클릭해서 새로운 노트북을 만들어주세요.



**02** 새 노트북은 Untitled[숫자].ipynb 이름으로 만들어지고 노트북에는 다음과 같이 빈 코드의 셀 하나가 들어 있습니다.

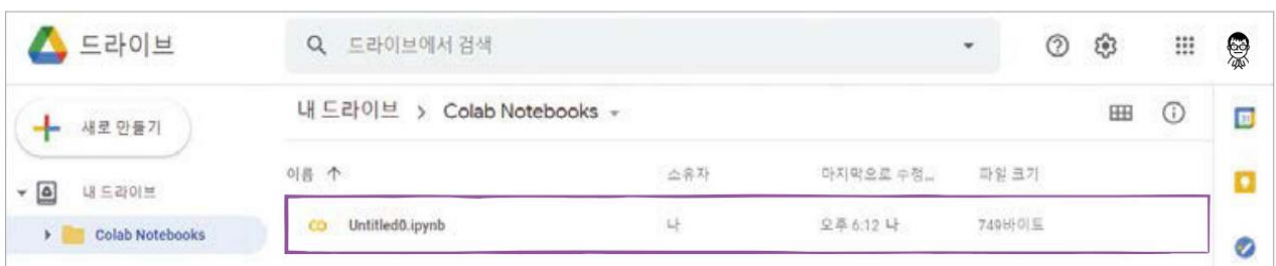




**03** 코드 셀에 'Hello World'를 출력하는 `print()` 코드를 작성하고 이 파일의 이름을 'Hello World'로 저장해 봅시다. 먼저 빈 코드 셀을 마우스로 선택하고 다음과 같이 입력한 다음 셀을 실행해 보세요. 코드 셀을 실행하려면 [Ctrl + Enter] 키(macOS 는 [cmd + Enter] 키)를누르거나 왼쪽에 있는 플레이 아이콘을 클릭합니다.

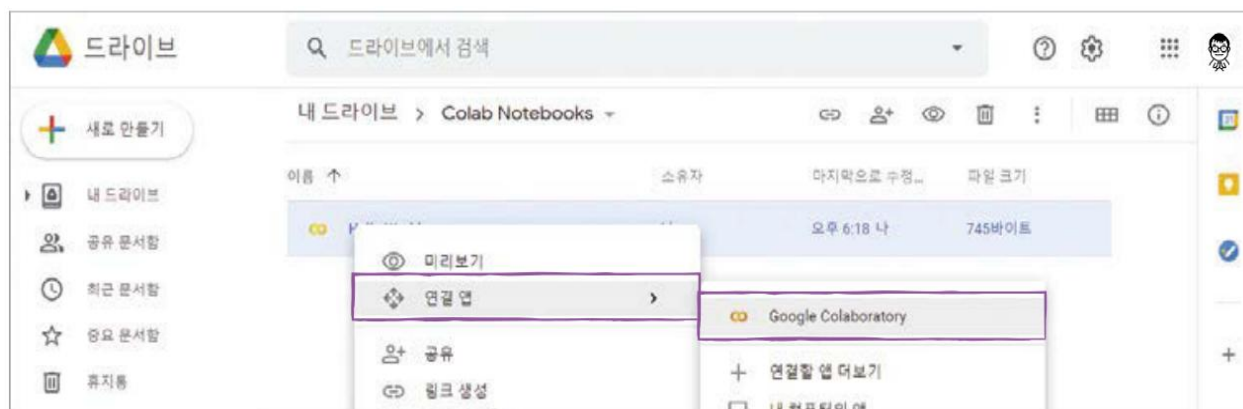


**04** 노트북은 자동으로 구글 드라이브 Google Drive 의 [내 드라이브]-[Colab Notebooks] 폴더 아래에 저장됩니다. 웹 브라우저에서 구글 드라이브(<https://drive.google.com>)로 접속해서 확인해 보세요.



**05** 이렇게 저장된 노트북을 코랩으로 불러올 수 있습니다. 코랩 노트북 화면에서 [파일]-[노트 열기]를 선택합니다. 팝업 창에서 [Google 드라이브]를 선택하면 [Colab Notebooks]에 들어간 노트북을 코랩에서 열 수 있습니다. 또는 구글 드라이브에서 코랩 노트북을 선택하고 마우스

오른쪽 버튼을 클릭하여 팝업 메뉴를 띄운 다음 [연결 앱]-[Google Colaboratory]를 선택하면 노트북을 코랩에서 열 수 있습니다.



지금까지 노트북을 만들고 저장하는 방법을 알아보았습니다. 코랩 노트북은 구글 드라이브에 보관되며 언제든지 다시 실행할 수 있습니다.

## GPU 세팅하는 방법

- ① CUDA 를 설치해야 합니다. CUDA 에서 본인 컴퓨터에 맞는 사양으로 설치하시면 됩니다.
- [CUDA Toolkit 11.5 Update 1 Downloads | NVIDIA Developer](#)
- ② cuDNN 파일을 컴퓨터에 붙여 넣어야 합니다.
  - cuDNN 에서 컴퓨터 사양에 맞는 버전을 다운 받습니다.
  - [NVIDIA cuDNN | NVIDIA Developer](#)
  - 다운 받은 파일의 압축을 풀어 cuda/bin, cuda/include, cuda/lib 의 폴더와 파일을 다음 경로에 붙여 넣습니다. (아래 경로 참조 하시어 컴퓨터 실제 경로에 맞도록 붙여넣습니다.)
    - C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\11.5
- 제어판 내 시스템 => 고급 시스템 설정 => 시스템 속성 의 고급 탭 => 환경 변수 에서 아래 경로를 추가합니다. 경로 추가 시 어떤 path 에서도 추가된 경로는 접근 가능해 집니다.
  - C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\11.5\bin
  - C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\11.5\libnvvp

## 윈도우에서 PyTorch 설치하는 방법

- GPU 를 범용적으로 사용하기 위해 CUDA 를 설치해줍니다.
- 딥러닝을 사용하기 위해 cuDNN 을 설치해줍니다. 여기까지는 앞의 과정을 그대로 따라하시면 됩니다.
- 그 다음 과정 부터는 conda 를 설치한 후 진행하는 방법과 pip 를 이용하는 방법이 있습니다.
- 1) conda 를 설치하려면 다음 링크를 통해 설치 파일을 받은 후 설치하면 됩니다.
  - <https://www.anaconda.com/products/individual>
- 2) pip 를 이용하여 설치하려면 virtualenv 를 이용하여 가상환경을 독립적으로 만들고 그 환경에서 설치하는 것을 권장드립니다. 이 방법을 이용하려면 다음 링크를 참조하시기 바랍니다.
  - [https://gaussian37.github.io/python-concept-initial\\_setting/](https://gaussian37.github.io/python-concept-initial_setting/)
- 위의 conda 설치 또는 pip 설치 과정이 끝났으면 pytorch 를 설치 할 가상 환경을 활성화 합니다.
- 1) conda 의 경우 다음과 같이 가상 환경을 활성화 합니다.
  - 가상 환경 생성 : conda create -n 가상환경이름 python=3.6
  - 가상 환경 활성화 : activate 가상환경이름

- 2) pip 의 경우 다음과 같이 가상 환경을 활성화 합니다.
  - 가상 환경 생성 : virtualenv 가상환경이름
  - 가상 환경 활성화 : 가상환경이름\Scripts\activate
- 위 두가지 방법 모두 가상 환경 활성화에 성공하면 command prompt 의 가장 앞에 (가상환경이름)이 추가된 것을 확인할 수 있습니다.
- 그 다음에는 pytorch, torchvision 을 설치해 보도록 하겠습니다. 반드시 다음 링크를 접속 하시기 바랍니다.
  - <https://pytorch.org/get-started/locally/>
- 위 링크에서 현재 설치하려는 환경의 옵션을 차례대로 선택합니다. 그러면 설치에 필요한 명령어가 생성됩니다. 그 명령어를 커맨드에 입력하여 pytorch 와 torchvision 을 설치하면 됩니다.
- 저의 경우 예를 들어 stable, windows, pip, python, cuda10.2 를 옵션으로 선택하였습니다. 이 때 설치 커맨드는 다음과 같습니다.
  - pip install torch==1.5.0 torchvision==0.6.0 -f [https://download.pytorch.org/whl/torch\\_stable.html](https://download.pytorch.org/whl/torch_stable.html)
- 실행이 정상적으로 되는지 확인하기 위해 다음 코드를 print(x) 까지 입력하였을 때, 출력이 정상적으로 되는 지 확인하면 됩니다.
- 여기서 .cuda() 명령어가 GPU 를 이용하는 명령어 입니다. 만약 CUDA, cuDNN 이 정상적으로 설치되지 않았다면 오류가 발생할 수 있습니다.
- 만약 오류가 발생하였을 경우 아래 명령어에서 torch.Tensor(3, 4).cuda() 대신 torch.Tensor(3, 4)을 사용하여 실행해 보시길 바랍니다. 이 경우 문제가 없다면 CUDA, cuDNN 이 설치가 잘못된 것입니다. 반면 이 경우에도 문제가 발생하였다면 pytorch 설치가 잘못된 것입니다.

```
import torch

x = torch.Tensor(3, 4).cuda()

print(x)

tensor([[ -5.5635e-24,  0.0000e+00,  4.4842e-44,  0.0000e+00],
        [          nan,  0.0000e+00,  2.6104e-09,  2.1162e-07],
        [ 1.6899e-04,  2.1240e+20,  1.0919e-05,  1.6969e-07]], device='cuda:0')
```

## colab 에서 PyTorch 사용하는 방법

- colab 에서는 기본적으로 PyTorch 가 설치되어 있지 않습니다.
- 따라서 매 세션마다 PyTorch 를 설치를 해주어야 합니다. 금방 설치되니 설치하고 사용하면 문제가 없습니다.
- 먼저 colab 메뉴의 **runtime** → **Change runtime type** 을 선택해서 **GPU** 를 선택해 줍니다.
  - 딥러닝을 사용하는 것이니 GPU 를 선택해 주어야 합니다.
- 아래 코드를 입력하여 PyTorch 를 설치해줍니다.

```
!pip3 install torch
```

```
!pip3 install torchvision
```

- 만약 colab 에서 google drive 의 파일 또는 디렉토리를 연결해서 사용하려면 아래 명령어를 입력하여 실행하면 연결된 계정의 google drive 를 사용할 수 있습니다.

```
from google.colab import drive
```

```
drive.mount('/content/gdrive')
```

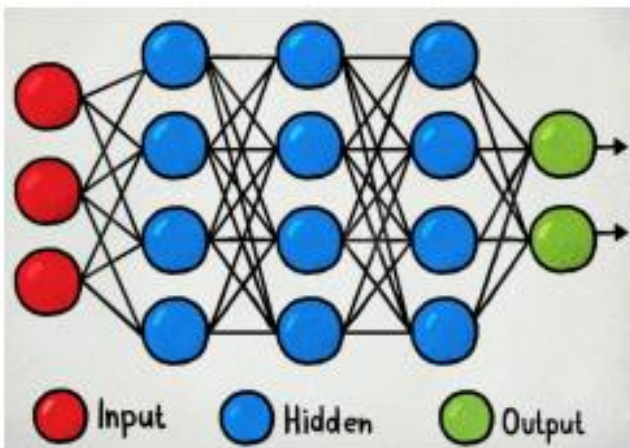
## PyTorch 에서 model summary 보는 방법

- 정의한 model 을 print 해서 보는 방법
- Keras 형태의 summary 를 보는 방법
  - pip install torchsummary 로 torchsummary 를 설치합니다.
  - summary(model.cuda(), input\_size) 를 실행합니다.

## 2. PyTorch 소개

### (1) 파이썬 기계 학습용 무료 라이브러리

- Facebook AI 연구팀이 2016년 공개
- 매뉴얼: <https://pytorch.org>
- 장점: GPU 지원의 원조, 효율적인 기울기 계산
- 약점: 상용화 지원 부족



- 인공지능 = 코딩
  - 1단계 : 사례를 통한 흥미 유발
  - 2단계 : PPT나 웹문서를 통한 이론 교육
  - 3단계 : 타인의 코딩 학습하고 실행해 보기
  - 4단계 : 직접 코딩하고 실행해 보기
  - 5단계 : 코딩 최적화 하기, 유사 문제 풀어보기
- 기계 학습 코딩 Package의 3대 천황
  - 텐서플로우 (Tensorflow)
  - 케라스 (Keras)
  - 아나콘다 (Anaconda)
  - 파이참 (PyCharm)

### 3. 머신러닝 기본 개념

#### 머신러닝

##### 1. 인공지능이란?

(1) 인공지능 = 도달하고자 하는 목적지

(2) 딥러닝

① 인공지능 > 머신러닝 > 딥러닝

② 가장 성공적인 결과를 내고 있는 인공지능의 핵심기술

(3) 머신러닝

① 데이터 기반 인공지능

② 머신러닝 + 딥모델 = 딥러닝

##### 2. 인공지능 구현 방법

###### 규칙 기반 인공지능

(1) 사용자가 컴퓨터에 데이터(규칙)를 입력하여  
입력된 데이터를 기반으로 해답 도출

① 장점: 새로운 지식에 대한 빠른 대응

② 단점: 입력되지 않은 규칙에 대한 결론 모색 불가

###### 데이터 기반 인공지능

(1) 사용자가 데이터를 입력하면 컴퓨터 스스로 규칙을 만들고  
해당 규칙에 기반하여 해답 도출

· 현재 각광받는 데이터 기반의 머신러닝 방법



### 3. 문제 해결 방법

#### 컴퓨터 공학의 문제 해결 방법

(1) 문제가 주어질 경우

- 문제 해결 알고리즘 디자인 → 프로그래밍 → 해답 도출

(2) 새로운 문제가 주어질 경우

- 새로운 알고리즘 디자인 → 프로그래밍 → 해답 도출

#### 머신러닝의 문제 해결 방법

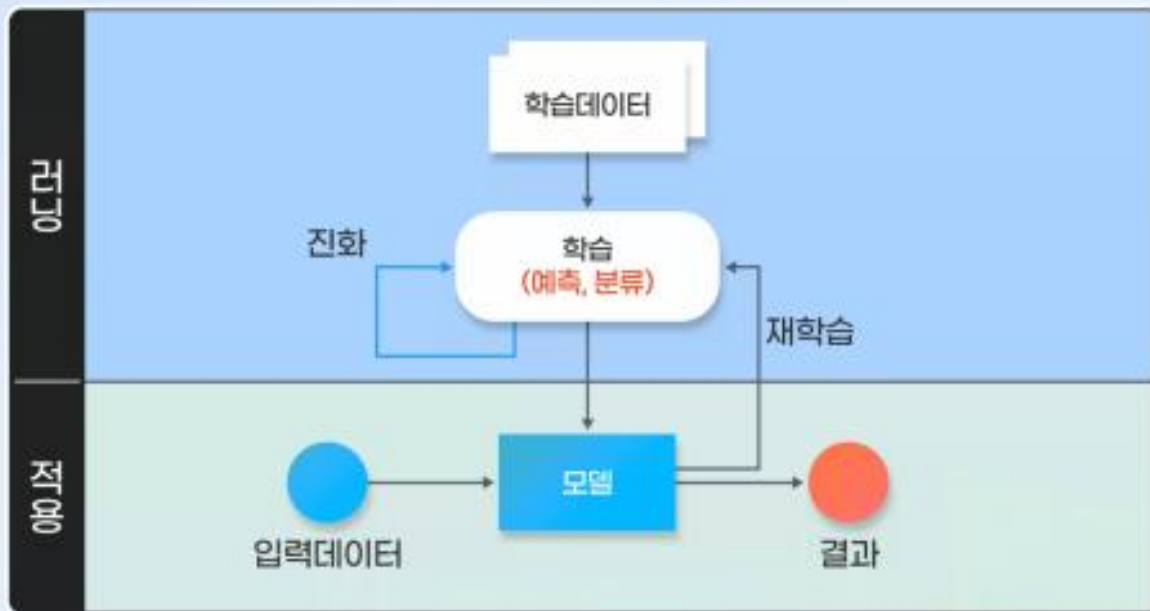
(1) 문제가 주어질 경우

- 데이터 수집 → 트레이닝(수집된 데이터 기반의 모델 선택 및 학습) → 해답 도출(학습된 모델 활용)

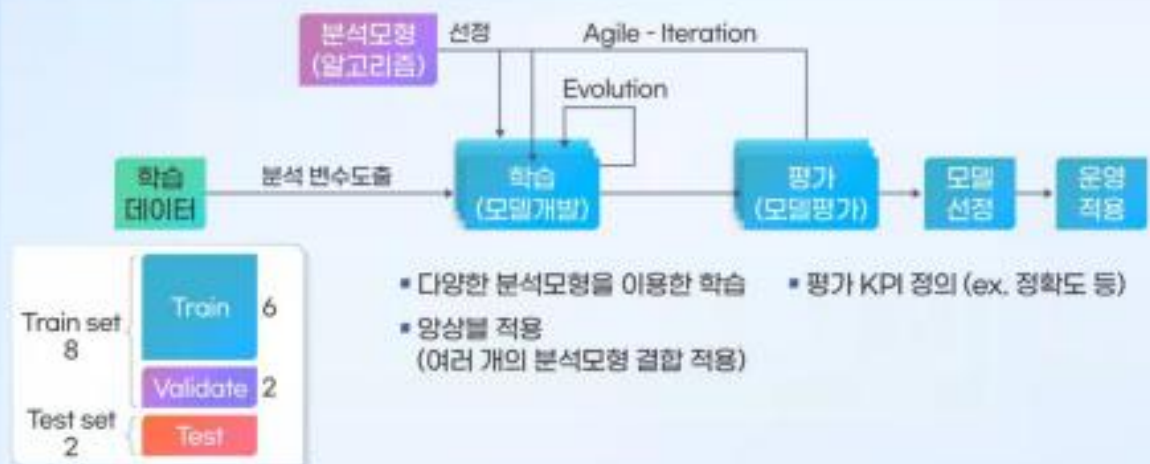
(2) 새로운 문제가 주어질 경우

- 문제 관련 데이터 수집 → 모델 재학습 → 해답 도출

## 머신러닝 학습 개념도



## 머신러닝 학습 단계



## 4. 머신러닝의 학습

### (1) 컴퓨터의 학습

- ① “문제, 데이터, 성능지표 세 가지가 주어졌을 때  
데이터 증가에 따라서 알고리즘 성능이 향상된다면 컴퓨터가  
학습하고 있는 것”(CMU 톰 미첼 교수)
- ② 머신러닝의 핵심은 컴퓨터 스스로 규칙을 찾아내는 것

### (2) 일반화(Generalization)

- ① 머신러닝 이해의 핵심 개념
- ② 새로운 데이터에 대한 정답 도출 능력이 일반화 능력의 척도
- ③ 일반화 능력이 머신러닝의 핵심 요소



## 머신러닝 3 대 요소

### 1. 머신러닝 구성 요소

- (1) 3대 요소: 데이터, 모델, 트레이닝 알고리즘
- (2) 추가 요소: 학습 과정, 일반화, 추론
  - 추론(Inference): 새로운 데이터 입력 시 해답을 추출하는 과정

### 2. 데이터

- (1) 정형 데이터(Tabular Data)
  - ① 데이터가 표 형태로 정형화되어 출력
  - ② 표의 행은 데이터 포인트 의미
  - ③ 표의 열은 데이터의 특징 표시
  - ④ feature가 특징으로, 사용자가 정의하여 각 단위가 다른 경우 발생
- (2) 비정형 데이터(Informal Data)
  - ① 이미지, 비디오, 텍스트, 음성, 오디오 데이터
  - ② 데이터의 특징이 불명확
- (3) 머신러닝의 비정형 데이터 분석 방법
  - ① 비정형 데이터의 특징 추출
  - ② 추출된 특징을 바탕으로 구별
- (4) 데이터 종류별 적합한 프로그램
  - ① 정형 데이터: 의사결정 나무 계열 프로그램
  - ② 비정형 데이터: 딥러닝 계열 프로그램

### 3. 모델

- (1) 입력 데이터를 받아 원하는 출력을 내어주는 것
- (2) 머신러닝 모델 = 수학의 함수
  - ① 파라미터: 함수의 모양을 결정하는 매개 변수
  - ② 파라미터 값: 데이터로부터 학습
  - ③ 학습: 데이터가 제공됐을 때 모델의 파라미터를 찾아주는 과정

### 4. 학습 알고리즘(Training Algorithm)

- (1) 알고리즘이란?
  - ① 일반적으로 문제 해결 과정을 모아 놓은 것
  - ② 머신러닝의 학습 알고리즘과는 다른 개념





### 1. 택사노미(Taxonomy)

- (1) 택사노미: 머신러닝의 분류 체계
- (2) 생성 모델(Generative Model): 비지도 학습
  - ① 생성형 AI: ChatGPT(텍스트 출력), Dall-E(이미지 출력)
  - ② 인식 과정
    - 두뇌에서 유사한 얼굴 생성 → 생성된 얼굴과 관측한 얼굴 비교
    - 일치 여부 판단 및 인식
- (3) 판별 모델(Discriminative Model): 지도학습
  - 해당 이미지를 분류하는 모델
  - 예: 사진 입력 후 강아지인지 고양이인지 분류하는 모델

### 2. 데이터 피드백 기준 택사노미(Taxonomy)

#### 지도학습(Supervised Learning)

- (1) 정답이 붙어 있는 지도학습의 훈련 데이터
- (2) 주어진 교본을 숙지하여 문제를 해결하는 방식
- (3) Instructional Feedback을 이용한 학습

#### 비지도 학습(Unsupervised Learning)

- (1) 입력 데이터만 존재
- (2) 데이터의 의미를 파악하는 과정을 학습하는 방식
- (3) 비지도 학습의 핵심은 Representation Learning

#### 강화 학습(Reinforcement Learning)

- (1) 모델이 취한 액션에 대해서만 피드백 제공

## 머신러닝의 알고리즘 형식 적용하기

### 1. 머신러닝의 개념

#### 1) 머신러닝의 기원

- 머신러닝(Machine Learning) : 데이터의 지능적인 행동에 따른 컴퓨터 알고리즘 개발 및 연구 분야
- 대용량에 따른 연산 능력과 데이터셋을 통계적 방법으로 분석하는데 필요한 원동력
- 데이터, 통계적 방법, 연산 계산 능력을 신속하게 처리하는 환경  
→ 머신러닝 발전 사이클



#### 2) 데이터마이닝

- 머신러닝과 상관관계인 데이터마이닝은 데이터베이스 측면에서 새로운 통찰력 관점
- 데이터마이닝
  - \* 실행 가능한 분야에서 체계적으로 추적
  - \* 문제 해결에 있어서 데이터마이닝은 컴퓨터를 통해 사람이 사용할 패턴을 검색하고, 가르치는데 집중함 (머신러닝은 컴퓨터의 데이터 사용법에 집중)
  - \* 대부분의 데이터마이닝은 머신러닝을 활용하지만 모든 머신러닝은 데이터마이닝 영역을 포함하지 않음

## 머신러닝의 알고리즘 형식 적용하기

---

### 3) 머신러닝 사용과 한계

- 머신러닝의 문제 해결 시 이해하는 능력은 상대적인 한계가 존재함
- 기계 학습은 순수한 지적 능력에 해당
- 빅데이터에서 패턴을 찾는 것은 사람보다 컴퓨터가 더 적합함
- 분석하는 것에 동기를 부여하고 결과의 의미와 실행 측면에서 사람이 필요함
- 기계는 질문의 이해와 유형별 질문 파악에 취약함
- 컴퓨터가 이해 가능한 방식의 질문이라면 파악 용이
- 머신러닝의 성공적 활용, 개선 영역, 피해 상황을 고려하여 파악해야 함

### 4) 머신러닝 성공 유형

- 전문적 지식의 대체보다는 지식의 증가 필요
- 인류의 생명 보호를 위해 전문 의사와 공유
- 스마트 홈, 자동차 제작에 엔지니어와 프로그래머 활용
- 사회학자는 사회적 지식을 쌓는 데 도움이 됨
- 정부기관, 기업, 병원, 과학연구소 등에서 사용
- 데이터 생성 또는 수집에서 데이터 이해에 적합한 머신러닝 알고리즘 활용

### 5) 머신러닝 사용 사례

- 최근 머신러닝 성공 사례
  - \* 재해의 보험 통계적 범위 추정
  - \* 사회 범죄 집중 지역 관리
  - \* 선거 결과의 예측
  - \* 고객 유형별 타겟 광고 개발
  - \* 신용카드의 불법성 거래 감시
  - \* 드론과 자율주행 자동차의 알고리즘 개발
  - \* 유전자를 이용한 질병 연구
  - \* 이상 기후 변화의 예측
  - \* 이메일의 스팸 구분
  - \* 에너지 소비 효율 최적화 전략



## 머신러닝의 알고리즘 형식 적용하기

---

### 2. 머신러닝의 한계

#### 1) 머신러닝 알고리즘의 한계

- 알고리즘
  - \* 데이터를 통해 미래 예측의 기초 패턴 식별
  - \* 학습 내용을 정확하게 파악하고, 파라미터의 포괄적 영역에 실제로 적용함
- 머신러닝
  - \* 인간 뇌의 상대적인 영역만 모방함
  - \* 학습한 파라미터의 범위 내에서 추정하고 적용 범위가 유연함
- 컴퓨터
  - 경험의 축적이 없음
  - 논리적인 단계를 상식적으로 추론하는 능력의 한계

#### 2) 머신러닝 광고의 문제점

- 광고 방문 이력을 분석하여 학습한 데이터마이닝 패턴을 웹 사이트 광고 제시에 활용 → 웹 사이트 광고에 사용자의 관심 상품에 대한 광고를 제시함
  - \* 고객은 광고 순환을 무한 반복하게 됨
  - \* 사람의 언어를 이해하거나 번역과 음성을 인식하는 등의 기능은 머신러닝 능력의 한계
  - \* 자신이 학습한 데이터만 중요한 사실로 인식함
  - \* 입력 데이터가 명확하지 않을 경우 컴퓨터는 인간과 동일하게 최적화된 추측만 가능함

#### 3) 머신러닝 윤리 인식

- 머신러닝 핵심 : 복잡한 데이터의 이해도 향상에 기여
- 기계적인 행동, 불합리한 소비자의 인식면에서 윤리적 문제 발생
- 컴퓨터를 통한 자동화 측면에서 예상치 못한 결과 초래
- 머신러닝 또는 데이터마이닝 사용자는 기술적인 면이 없어 불성실할 가능성이 있음

## 머신러닝의 알고리즘 형식 적용하기

### 4) 머신러닝 발전 속도

- 머신러닝의 확산 및 발전 : 관련 법률과 사회 규범은 매우 불확실하고 유동적임
- 일반 대중의 개인정보를 침해하지 않도록 데이터 분석에 주의
- 머신러닝의 분석 결과 적용에 앞서 상식적인 면 반영  
(건강 데이터 등의 민감한 정보)
- 머신러닝을 통해 시나리오 예측이 가능하므로 발견한 패턴의 노출면에  
신중한 접근 필요

### 5) 머신러닝의 부적절한 측면

- 머신러닝의 알고리즘 보편화를 위한 컴퓨터는 성, 인종차별 등 부정적이고  
고정화된 관념을 포함한 인류의 부적절한 측면을 학습할 수 있음
- 머신러닝의 악의적인 공격은 치명적
- '적대적 공격'과 악의가 없는 의도에 대한 잘못된 인식으로  
치명적인 사고 유발 가능
- 실생활에서 잘못된 남용이 없도록 윤리 의식과 주의 필요

## 머신러닝의 알고리즘 형식 적용하기

---

### 3. 기계학습 방법

#### 1) 기계학습 방법

- 톰 미첼(Tom M, Mitchell)의 공식적인 머신러닝 정의
  - \* “미래 경험을 통한 성능이 향상되면 기계는 자신의 경험을 활용할 수 있을때마다 학습한다”
- 미래의 실행에 따라 변환은 완전히 무시됨
- 학습은 실제 실행하는 데 어려움이 있음
- 인간의 뇌는 자연적인 학습 능력을 가지지만, 컴퓨터는 필요한 조건에 따라 명시적으로 학습
- 머신러닝 알고리즘은 이해와 구별을 위해 구현하는 데 도움을 줌

#### 2) 머신러닝과 인간학습 비교

- 인간과 기계의 기본적인 학습 과정은 유사함
- 학습 과정은 네 개의 구성 요소로 구분
  - ① 데이터 셋 저장소(Data Storage) : 기억(memory), 관찰(Observation), 소환(Recall)을 통한 미래 추론은 적합하고 현실적인 토대를 바탕으로 함
  - ② 추상화개념(Abstraction) : 데이터의 포괄적인 표현과 개념적인 변환
  - ③ 보편일반화(Generalization) : 데이터를 활용해 지식과 추론적인 면을 생성하고, 새로운 환경에서 실행
  - ④ 결과 평가(Evaluation) : 학습을 통해 지식의 효율적인 면을 측정하여 개선 사항을 표시하고, 피드백을 메커니즘으로 제공

#### 3) 학습과정

- 사람은 마음을 통해 기억, 추론, 유도 과정을 거쳐 직관적 학습 가능
- 사람에 따른 차이는 주관적 관념에 따라 기인됨
- 학습적인 지식은 미래의 실행 측면에서 관찰, 전환, 활용 데이터를 ‘과학’으로 간주

## 머신러닝의 알고리즘 형식 적용하기

---

### 4. 실전 머신러닝

#### 1) 실전 머신러닝 데이터

- 머신러닝 알고리즘의 단계적인 배포 : 데이터 수집을 통한 데이터 탐색과 준비
- 데이터 수집
  - \* 텍스트 파일, 스프레드 시트, 데이터베이스 등의 사용할 학습 자료를 수집한 후 알고리즘 실행 가능한 지식 생성
- 데이터 탐색과 준비
  - \* 머신러닝 프로젝트의 고품질 여부는 데이터의 품질에 따라 좌우됨
  - \* 데이터 탐색 작업은 데이터 간의 미묘한 차이에 따른 파악 필요
  - \* 학습 과정에 적합한 데이터의 준비와 추가 작업 필요
  - \* 데이터의 교정 또는 정리
  - \* 불필요한 데이터 제거
  - \* 학습자의 기대에 적합한 데이터 코드화 작업

#### 2) 실전 머신러닝 모델

- 모델 훈련
  - \* 분석용 데이터를 준비하여 데이터로 학습
  - \* 머신러닝 작업 : 적합한 알고리즘 선택
  - \* 알고리즘 : 모델화 한 형태로 데이터 표현
- 모델 평가
  - \* 머신러닝 학습 모델 알고리즘은 자신의 경험적인 학습 평가 면에서 중요
  - \* 모델 종류에 따라 테스트 데이터 셋 모델의 정확도 면 평가
  - \* 특화된 성능 척도 면 개발 필요
- 모델 개선
  - \* 모델의 성능 향상은 고급 전략 활용
  - \* 다른 종류의 모델로 전환할 가능성 있음
  - \* 추가 데이터로 데이터 보충 또는 추가 준비 작업 수행이 필요할 수 있음

## 머신러닝의 알고리즘 형식 적용하기

---

### 3) 예측된 작업 배포

- 모델의 정상적인 작동은 예측된 계획의 작업용으로 배포
- 예측용 데이터 제공, 마케팅 또는 리서치의 통찰 데이터 생성, 작업의 자동화 모델로 활용
- 모델의 성공 여부는 차세대 학습자의 훈련을 위한 데이터로 활용

### 4) 입력 데이터 형식

- 머신러닝에는 학습 알고리즘을 입력 데이터에 적용하는 작업이 포함됨
- 머신러닝을 문제에 적용하기 전에 입력 데이터 셋을 구별하고 용어 의미 이해하는 것이 중요
- 예시 : 속성을 포함한 관측 데이터의 인스턴스
- 특징 : 학습에 활용할 수 있는 예시 데이터의 기록된 속성 또는 특성

### 5) 비구조화 데이터

- 처리 가능한 입력 데이터 면의 복잡도에 따라 사람과 기계로 구분
- 사람
  - \* 텍스트, 그림, 소리 등의 비구조화 데이터 처리 가능
  - \* 방대한 속성을 포함한 관측치 처리에 유동적
  - \* 특징이 약한 관측치 처리에 유연함

### 6) 데이터 셋 특징

- 수치 : 숫자 형태로 측정된 특성
- 범주의 집합 유형 : 범주 또는 명목으로 구분
- 범주형 변수 : 서열
- 서열 목록의 범주 : 명목 변수
- 개수와 종류는 머신러닝 알고리즘을 결정함
- 특징의 표현 방법이 중요

## 머신러닝의 알고리즘 형식 적용하기

---

### 5. 머신러닝의 알고리즘 형식

#### 1) 머신러닝 지도학습

- 머신러닝 알고리즘 : 목적에 적합한 다양한 유형으로 구분
- 데이터를 통해 머신러닝 알고리즘의 유형 측면에서의 이해 필요
- 예측 모델 : 데이터 셋의 값으로 새로운 데이터 값 예측
- 학습 알고리즘 : 목표의 특징 사이의 관계를 찾는 모델링
- 예측 모델을 통한 사건의 예측은 불필요함
- 지도학습 : 학습 방법의 명확한 지침에 따른 예측 모델의 훈련 과정

#### 2) 머신러닝 목표 출력

- '지도'란 인간 개입의 의미보다 목표 값이 학습자가 원하는 작업에 따른 학습 정도를 의미
- 보다 형식적인 지도학습 알고리즘으로 데이터 셋 함수로 최적화
- 목표 출력의 특징적인 데이터 값에 따른 조합 탐색
- 범주를 예측하고 지도 머신러닝 작업으로 분류

#### 3) 머신러닝 분류예측

- 범주형 변수, 서열 목록의 범주를 목표형으로 구분
- 클래스 : 다수의 레벨로 구성
- 레벨 : 순위 또는 비순위로 구분
- 머신러닝에서 다양하게 활용되며, 유형별 입력 데이터에 다양한 분류별 알고리즘을 형성함
- 지도 학습자의 활용
  - \* 급여, 실험 데이터 값, 점수 결과, 개수 등의 숫자 데이터를 예측하는 데 활용

## 머신러닝의 알고리즘 형식 적용하기

### 4) 머신러닝 선형회귀 모델

- 수치 값의 예측 형태는 선형 회귀 모델을 입력 데이터에 따라 구성
- 회귀 모델 : 관계의 불확실성과 크기를 모두 포함된 형태로 입력
- 목표 사이의 연관성을 정량화 하기 때문에 예측 면에서 광범위하게 활용 가능

### 5) 머신러닝 서술모델

- 서술모델은 데이터 요약, 통찰 작업에 활용
- 관심 목표를 예측하는 모델과는 다르게 서술모델은 다수의 특징 중 하나의 특징만 유효함
- 실제 학습 목표가 없으므로 서술 모델의 훈련과정을 '비지도학습'이라 칭함
- 서술모델의 응용에는 어려움이 있으나 데이터마이닝에서 자주 사용함

### 6) 머신러닝 패턴모델

- 패턴 발견의 서술형 모델링 작업은 데이터에 유용한 연관 검색에 활용
- 패턴 발견은 거래 데이터와 구매 데이터 분석에 사용
- 판매자는 구매품목의 식별을 통해 학습된 정보를 마케팅 전술에 반영한 후 수익 창출로 활용



## 머신러닝의 알고리즘 형식 적용하기

---

### 7) 머신러닝 동질 그룹

- 데이터 셋을 동질 유형 단위로 그룹화 → 서술형 모델링 작업을 통해 군집화
- 군집화의 식별 : 유사한 행동 또는 비슷한 인구학적 정보를 개인 그룹 형태로 식별
- 광고 캠페인으로 특화 시키기 위해 전문적이고 세분화된 분석에 활용
- 기계로 진행된 클러스터 해석이 있더라도 전문화된 해석은 사람의 판단이 필요함

### 8) 메타학습 알고리즘

- 머신러닝 알고리즘 분류
  - \* 특정 작업에 국한되지 않고 효과적으로 '학습하는 방법'을 학습하는 것에 주력
  - \* 메타학습 알고리즘은 새로운 정보 분석을 위해 기존 학습 결과를 활용함
  - \* 머신러닝과 강화학습은 시간의 흐름에 따라 진화하는 알고리즘을 포함함
  - \* 메타학습 : 문제 또는 알고리즘의 성능이 정확할 때 유용
  - \* 적대적 학습 : 모델의 장점보다 약점을 학습하여 미래의 성능을 강화함
  - \* 악의적 공격 방어에 적합
  - \* 고성능 컴퓨터 또는 클라우드 컴퓨팅 환경에서 대용량 데이터 셋을 모델링 함



## 입력 데이터 유형으로 알고리즘 매칭하기

### 1. 머신러닝 알고리즘의 일반적인 종류

#### 1) 머신러닝 알고리즘의 종류

- 머신러닝 알고리즘 분류
  - \* 4종류의 학습 작업 구조인 수치 예측, 분류, 군집화, 패턴 감지 중에서 프로젝트에 적합한 작업 결정
- 작업 알고리즘 선정으로 유도
- 패턴 감지 수행 후 연관된 규칙 활용

#### 2) 유사 데이터와 알고리즘 매칭

- 유사한 군집화 문제
  - \* K-평균 알고리즘 활용
  - \* 수치 예측 활용 수단 : 회귀 분석 또는 회귀 트리
  - \* 분류에 적합한 학습 문제와 최적화된 분류기를 일치시키기 위해 많은 분석 필요

#### 3) 데이터와 알고리즘 차이

- 알고리즘의 다양한 차이
  - \* 분류 문제 구조 : 의사 결정 트리 유형은 이해하기 쉬운 모델로 구성
  - \* 신경망 모델 : 해석 면에서 고난이도로 구성
  - \* 신경망을 통해 잘 예측한 경우도 예측에 대해 설명할 수 없다면 응용 불가

## 4. 데이터 전처리

### 데이터 전처리 정의 및 목적

#### 1. 데이터 전처리란?

- 원시 데이터(raw data)를 분석에 적합한 형태로 변환하는 과정

#### 2. 데이터 전처리 목적

- 데이터 품질 향상
- 분석 또는 모델링 과정에서의 오류 최소화 및 정확한 결과 도출

### 데이터 전처리 작업들

#### 1. 정제(Cleansing)

- 결측치 처리
- 이상치 처리
- 중복데이터 제거

#### 2. 통합(Integration)

- 분석을 위한 여러 데이터 통합

#### 3. 변환(Transformation)

- 데이터의 척도를 맞춰주는 작업
- 스케일링
- 정규화
- 범주형 데이터 인코딩

#### 4. 데이터 축소

- 차원(변수)축소
- 샘플링

## 5. 특징 선택 및 생성

- 모델 성능 향상을 위한 주요 특징(변수) 선택 및 생성

## 결측치 처리

### 1. 결측치 처리 개념

- 데이터셋에서 누락된 값(Missing values)을 제거하거나 적절히 대체하는 작업

### 2. 결측치 식별 방법

- 단순 탐색
- 시각화
- 결측치 패턴 분석

### 3. 결측치 처리 방법

- 삭제
- 평균/중앙값/최빈값 대체
- 전진/후진 채우기
- 임의 값 대체
- KNN 대체
- 회귀 대체
- 다중 대체

## 이상치 처리

### 1. 이상치 처리 개념

- 데이터셋 내에서 다른 데이터 포인트들과 현저히 차이가 나는 값

### 2. 이상치 식별 방법

- IQR 방법
- 박스플롯 사용
- 상한선/하한선 설정

### 3. 이상치 처리 방법

- 이상치 제거
- 평균/중앙값 대체
- 최빈값 대체
- 로그 변환
- 제곱근 변환
- 클리핑
- 회귀 모델 사용
- KNN 대체

## 데이터 변환

데이터 전처리를 위한 데이터 변환을 해봅시다.

**데이터 변환**이란 데이터 분석, 머신러닝 모델링,  
시각화의 효과성을 높이기 위해  
데이터를 다른 형식으로 변환하는 과정을 의미합니다.

**방법** 데이터 분포정상화, 데이터 범위 표준화,  
이상치 영향 감소 등의 목적이 있습니다.

## 5. 지도학습 알고리즘

### 선형 회귀

#### 1. 지도학습

(1) 훈련 데이터(Training Data)는 N개의 Labeled Data 형태

$$D = \{(x_1, y_1), \dots, (x_N, y_N)\}$$

$x$  = 입력 데이터,  $y$  = 레이블

① 회귀(Regression):  $y \in R$  (실수)

② 분류(Classification):  $y \in \{0, 1\}$

#### 2. 선형 회귀(Linear Regression)

(1) 집의 크기와 가격 사이의 관련성 모델링 : 3개의 샘플 취득 시

$$D = \{(x[1], y[1]), (x[2], y[2]), (x[3], y[3])\}$$

(2) 두 개의 파라미터를 갖는 선형 모델

$$y[n] = w_1 \times [n] + w_0, \quad n = 1, 2, 3$$

(3) 3개의 샘플을 선형 모델에 대입 : Linear Algebraic Equation

① 수식은 3개, 미지수는 2개

$$y[1] = w_1 \times [1] + w_0,$$

$$y[2] = w_1 \times [2] + w_0,$$

$$y[3] = w_1 \times [3] + w_0$$

(4) 차선택: 등호를 '비슷하다'로 변경

$$e[1] = y[1] - w_1 \times [1] - w_0,$$

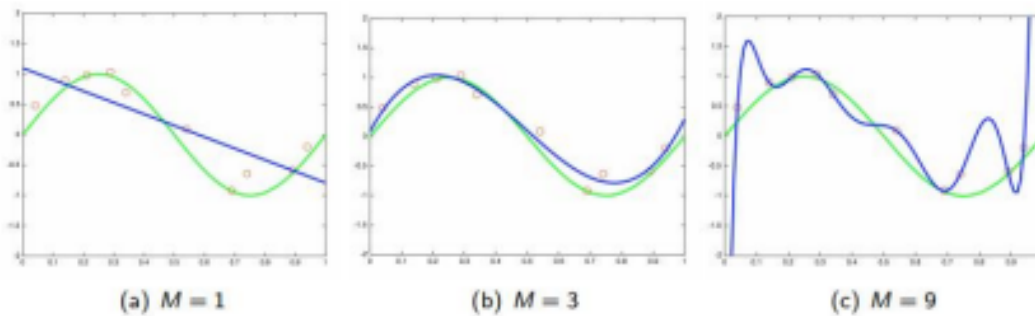
$$e[2] = y[2] - w_1 \times [2] - w_0,$$

$$e[3] = y[3] - w_1 \times [3] - w_0$$

(5) 에러 제곱의 합을 최소화하는 미지수 계산

$$\arg \min e[1]^2 + e[2]^2 + e[3]^2$$

### 3. 다항 회귀(Polynomial Regression)



(1)  $\hat{y}[n] = w_0 + w_1x[n] + w_2x^2[n] + \cdots w_Mx^M[n]$

- ①  $M$  : 사용자 지정 값, 하이퍼파라미터(Hyperparameter)
- ②  $M=3$  적합,  $M=9$  과적합(Overfitting)

(2) 정규화(Regularization) 기법

- ① 과적합이 발생하면 가중치(Weight) 값 증가
- ② 이때 가중치 값이 특정 값 이상 커지지 못하도록 제한하는 방식

### 4. 리지 회귀(Ridge Regression)

(1) 에러 제곱의 합을 최소화하는 가중치를 찾는 방법

$$e[n] = y[n] - (w_0 + w_1x_1[n] + \cdots + w_Mx_M[n])$$

(2) 가중치 값이 특정 값 이상 커지지 못하게 제한 조건을 두는 것으로 선형 회귀에서 가장 많이 사용

$$\arg \min_{w_0, w_1, \dots, w_M} (e[1]^2 + \cdots + e[N]^2) + \lambda(w_0^2 + w_1^2 + \cdots + w_M^2)$$

(3) 제한 조건을 두면 람다(Hyperparameter) 등장

- 에러와 정규화 사이의 밸런스 결정 및 사용자 지정

### 1. 로지스틱 회귀(Logistic Regression)

(1) 이진 분류(Binary Classification) 문제에 가장 먼저 사용되는 알고리즘

- ① 이진 분류의 아웃풋은 비연속 값
- ② 확률 값  $P(y = 1 | x)$  출력 기대

(2) 선형 회귀 모델에서 출력값=실숫값

(3) Logistic Function을 이용해 실숫값을 0과 1사이의 확률 값으로 변환

$$P(y = 1 | x) = \frac{1}{1 + e^{-\text{선형 모델 출력값}}}$$

### 2. 로지스틱 함수

(1)  $g(z) = \frac{1}{1 + e^{-z}}$

$g(z)$  = Functional Form

$z$  = 입력값(연속적인 값)

(2) 아웃풋 값은 0에서 1 사이

### 3. 로지스틱 회귀 모델

$$\hat{y} = \frac{1}{1 + \exp(-(w_0 + w_1x_1 + \cdots + w_Mx_M))}$$

### 4. 크로스 엔트로피 에러(Cross-Entropy Error)

(1)  $J = -\sum_{n=1}^N y[n] \log \hat{y}[n] + (1 - y[n] \log(1 - \hat{y}[n]))$

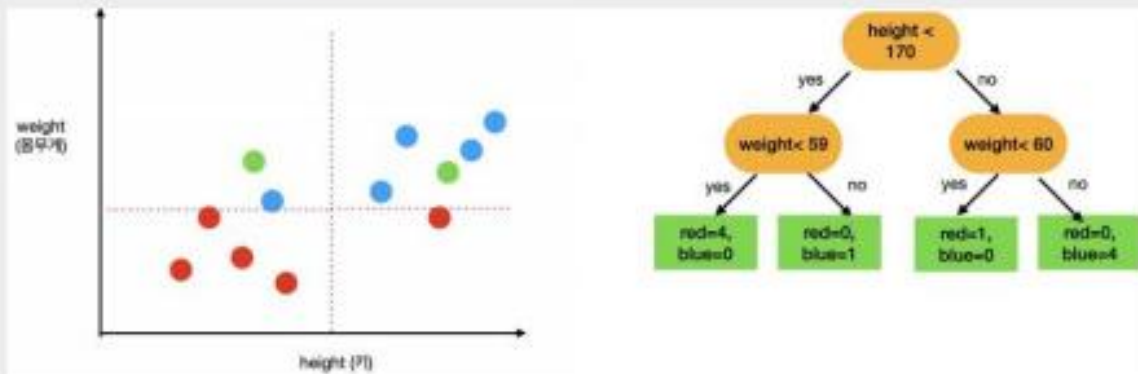
(2) 크로스 엔트로피 에러를 최소화하는 가중치  $w_0, w_1, \dots, w_M$  을 특정 알고리즘 또는 최적화 방법에 의해 추출



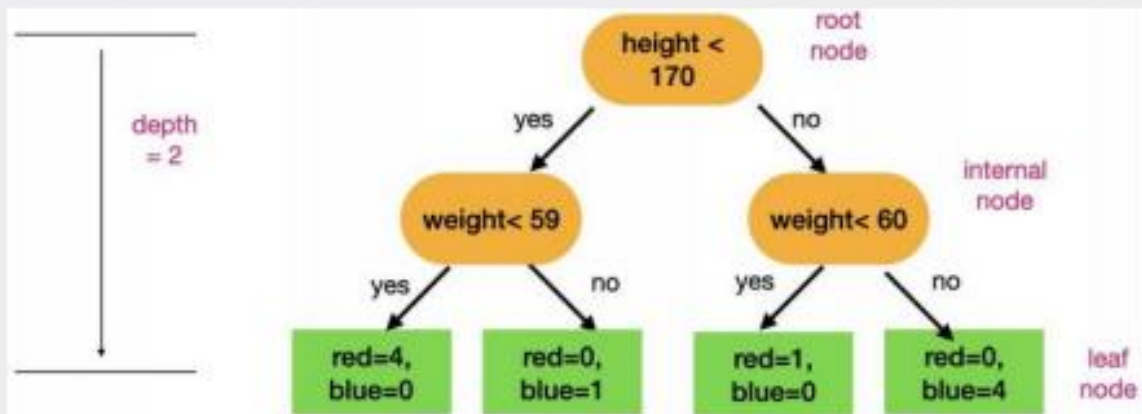
## 의사결정 트리

### 1. 의사결정 나무(Decision Tree)

#### 의사결정 나무 예시



#### 의사결정 나무 용어



### 2. 스플리팅(Splitting)

- (1) 핵심은 특정 클래스만 모여 있게 나누는 것
- (2) 리프 노드 도달 시 특정 클래스의 데이터만 분류되는 게 이상적
- (3) 메저(Impurity) : 스플리팅 시 클래스 혼합 정도
  - 메저를 이용하여 자동 스플리팅한 후 최적의 경우 자동 검색

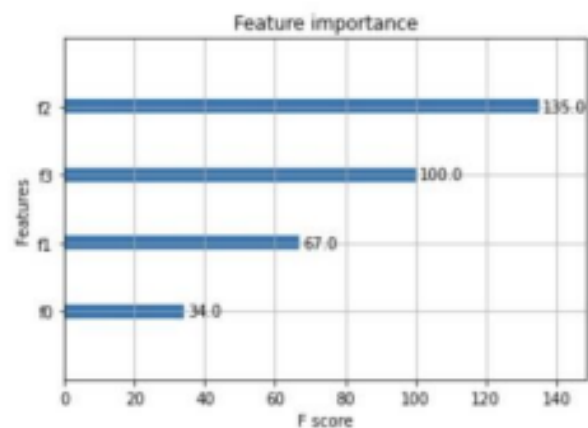
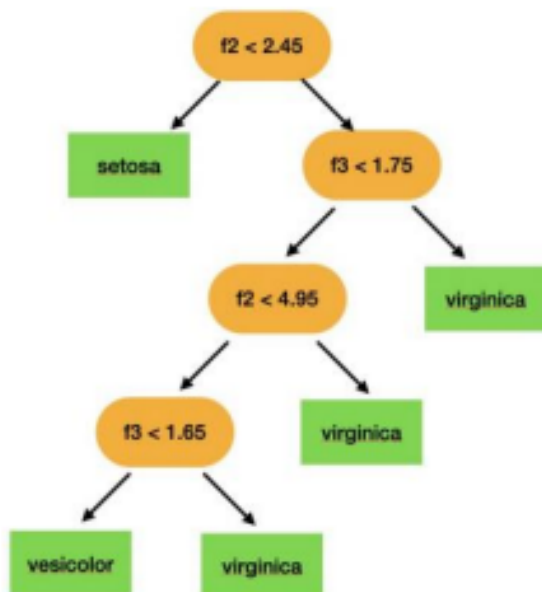
### 3. 아이리스 데이터

(1) Tabler Data → Classification 트리

- 리프 노드에 특정한 클래스의 데이터 포인트만 갖도록 지정

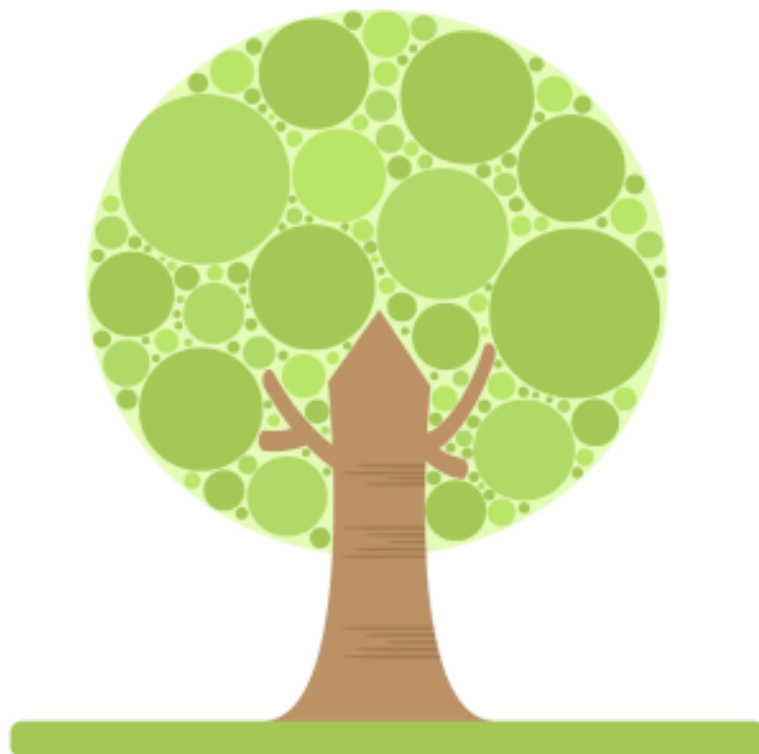
(2) Classification 트리 → Feature Importance Plot

- ① Feature가 미치는 영향력 측정
- ② Feature의 중요도 판단 필요시 Feature Importance Plot 통해 확인



## 4. 의사결정 나무의 앙상블

- (1) 하나의 의사결정 나무로는 성능이 아주 우수한 편이 아님
  - 분류(Classification) 문제,  
회귀(Regression) 문제 해결 시 사용
- (2) 의사결정 나무를 여러 개 만들어도 각기 개별적인 성능을 내야 함
- (3) 다양한 의사결정 나무 생성의 핵심은 Randomness 활용
  - ① Bagging(Bootstrapping + Aggregation): Bootstrap Sample을 이용, 여러 개의 의사결정 나무를 병렬 생성
  - ② Boosting: 의사결정 나무를 순차적으로 만들어가며 오류를 줄여가는 방식

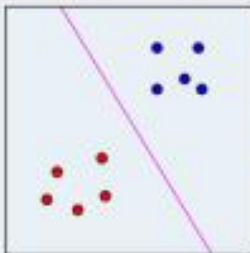


## 6. 비지도학습 알고리즘

### 데이터 군집화

#### 1. 지도학습과 비지도학습

##### 지도학습

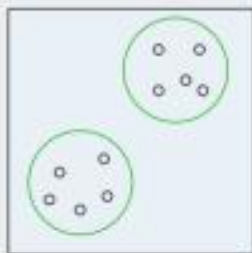


(1) 색상에 따라 레이블 구분

(2) 레이블 정보를 이용해 Decision Boundary 설정

##### 비지도학습

(1) 훈련 데이터(Training Data)는 N 개의 Unlabeled Data 형태로 제공:  $D = \{(x, \dots, x_N)\}$



① 레이블 정보 없음

② 유사한 데이터 간 그룹핑

#### 2. 클러스터링(Clustering) 기술 적용

(1) Targeted Marketing : 성향이 비슷한 소비자끼리 그룹핑하여 그룹별로 차별화된 마케팅을 하는 기법

(2) Image Clustering : 제품의 이미지 결함 처리 필요시 → 클러스터링 기법을 사용해 이미지 결함 군집화 → 군집을 바탕으로 데이터 레이블링

(3) Community Detection : 소셜 미디어 내 비슷한 성향의 유저를 그룹핑하는 기법

### 3. 군집화를 위한 알고리즘의 결정사항

- (1) 각 데이터 포인트가 속하는 클러스터(Cluster)
- (2) 각 클러스터의 센터(Center)

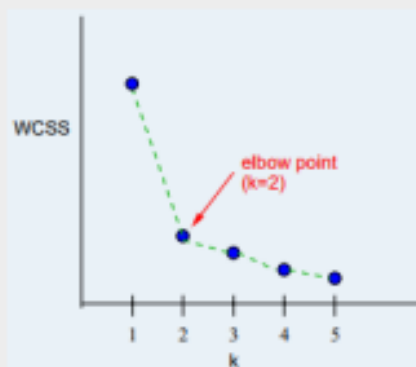
### 4. $k$ -means 알고리즘

- (1) 초기 설정
  - ① value of  $k$
  - ②  $k$  centers :  $u_1, \dots, u_k$ 
    - $k$  = 클러스터의 개수, 클러스터의 센터 값
    - 사용자 지정 필요
- (2) Assignment Step: 각각의 데이터 포인트와 가장 가까운 클러스터 센터 간의 거리 계산
$$\|x_n - u_k\|^2$$
  - ① 가까운 거리에 따라 데이터 포인트 그룹핑
  - ② 업데이트된 센터와의 거리 재계산
- (3) Update Step: 그룹들의 센터 재계산
$$u_k^{new} = \frac{1}{N_k} \sum_{n \in C_k} x_n$$
  - 그룹별로 새로운 센터 재계산(센터=평균값)
- (4) 수렴될 때까지 두 단계를 반복 수행

## 5. $k$ -means 알고리즘의 약점

- (1) 초깃값이 결과에 미치는 영향이 큼
- (2) 클러스터가 컨벡스 형태일 때만 정상 동작  
: 유클리디안 거리 사용 시 동작 어려움
- (3)  $k$ 값 지정 방법에 대한 명확한 이론 부재

## 6. Elbow Method : $k$ 값 정하기



- (1) 여러 개의  $k$  값에 대한 WCSS(Within-Cluster Sum of Squares) 값을 구해  $k$  값 결정
- (2) 각각의 데이터 포인트와 센터 간 거리의 총합을 이용해  $k$  값 결정

## 7. Two Moons Data



클러스터 형태가 컨벡스가 아님:  $k$ -means 클러스터링 적용



## 잠재 공간과 임베딩

### 1. 잠재 공간(Latent Space)

- (1) 잠재 변수: 관측 데이터의 의미를 함축·요약하는 변수
- (2) 잠재 공간: 잠재 변수가 내재된 공간
  - ① 잠재 공간은 관측 데이터 공간보다 간결한 확률 분포로 정돈
  - ② 머신러닝에서도 잠재 변수 모델(Latent Variable Model) 사용

### 2. 생성 모델(Generative Model)

- (1) 디코더(Decoder): 잠재 변수 → 데이터
  - 디코더 설계가 생성 모델의 핵심
- (2) 인코더(Encoder): 데이터 → 잠재 변수

### 3. 임베딩(Embedding)

- (1) 딥러닝의 임베딩 : 보통 데이터를 잠재 공간 또는 시맨틱(Semantic) 공간으로 매핑 시켜주는 방식
- (2) 장점
  - ① 데이터의 의미 파악 수월
  - ② 좋은 특징 추출 가능

## 4. PCA(Principal Component Analysis)

- (1) 차원 축소(Dimensionality Reduction) 기술로 사용
- (2) 공간 특징, 통계적 특징 추출 기술
- (3) 복원 에러를 최소화하는 방향으로 리니어 프로젝션을 찾는 기술
  - 리니어 프로젝션의 구성: 데이터 공분산 행렬에서 가장 큰 D개의 Leading Eigenvectors로 구성

## 5. t-SNE(Stochastic Neighbor Embedding)

- (1) 데이터를 2차원 또는 3차원으로 시각화하는 기술
- (2) PCA 같은 선형 변환 기술 사용 시 데이터 릴레이션 깨짐 발생





### 1. 행렬 분해(Matrix Decomposition)

(1) 레이블 없는 입력 데이터의 행렬

$$X = [x_1, \dots, x_N] \in R^{D \times N}$$

(2) 데이터 행렬  $X$ 를  $U$ 와  $V$ 의 곱으로 분해

$$X \approx UV^T, U \in R^{D \times d}, V \in R^{N \times d}$$

①  $U, V$ 는 알고리즘이 도출

②  $U, V$ 는 잠재 변수

(3) SVD(Singular Value Decomposition)

$$X = U\Sigma V^T$$

(4) NMF(Nonnegative Matrix Factorization)

$$X = UV^T (X, U, V > 0)$$

$$\arg \min_{U, V} \|X - UV\|_2^2 \text{ (subject to } U, V > 0)$$

①  $X$ 와  $U, V$ 의 곱의 차이가 최소화되는  $U, V$  찾기

- Alternating Least Squares 기술
- Projected Gradient Descent 방법
- Multiplicative Updates 기술

## 2. 행렬 분해(Matrix Decomposition)의 특징

- (1) 대표적인 특징은 보이는 이미지를 계산하는 기술
  - ① Data Matrix를 2개의 인자 행렬로 분해
  - ② 분해된 Matrix에 데이터 복원
  - ③ 행렬 분해 결과를 예측
- (2) 각각의 아이템에 대한 유저의 긍정, 부정 반응을 예측하는 기술
  - 예: OTT 영화 추천 기술

## 3. Representation Learning

- (1) 머신러닝 모델이 구별하기 위한 Decision Boundary 추출
  - ① 직선이 아닌 형태의 Decision Boundary 추출 어려움
  - ② 코디네잇 변경 시 직선에 의해 구별 가능
- (2) Classification, Prediction Model 제작에 효과적
- (3) 비지도학습의 핵심